Develop AI Agents on Azure

Agents, tools, orchestration, and interoperability

Rick Beerendonk
rick@oblicum.com

glasspaper oblicum

© Rick Beerendonk

What You'll Learn

- Explain what turns a chat model into an agent
- Tell conversation state, knowledge, and memory apart
- Give an agent tools, and decide who is allowed to run them
- Connect an agent to company knowledge and to Microsoft 365
- Choose an orchestration shape when one agent is not enough
- Recognise when to use MCP and when to use A2A
- Find which part failed when an agent gives a bad answer

© Rick Beerendonk

Problem: A Chat App Cannot Do Anything

A chat model produces text. Ask it to "book the meeting room" and it produces a *description* of booking a meeting room.

- ❌ It cannot look up today's data
- ❌ It cannot call your systems
- ❌ It stops after one answer, even if the job needs five steps

Need: something that can think, act, look at the result, and keep going until the job is done.

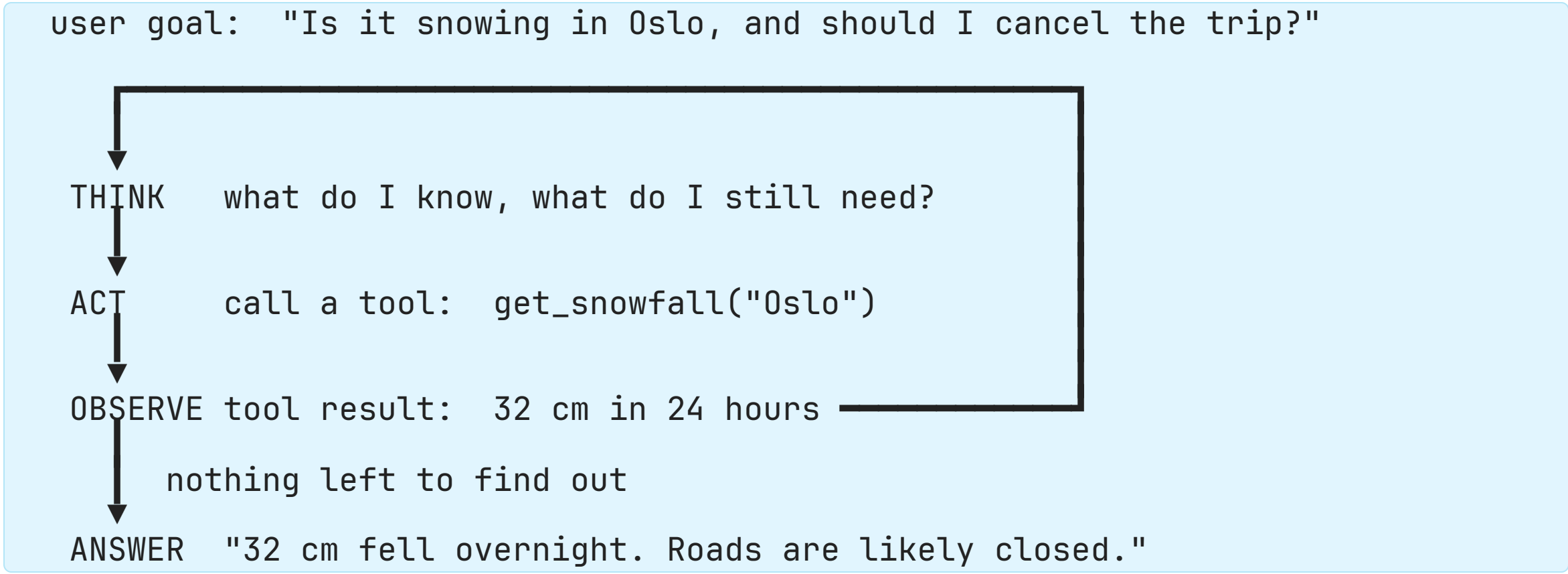
Agent = model + instructions + tools + a loop.

The loop is the part people forget. It is the whole trick.

glasspaper oblicum

© Rick Beerendonk

The Agent Loop



The model decides when to stop looping. Instructions, tool descriptions, and approval settings are how you keep that decision under control.

glasspaper oblicum

© Rick Beerendonk

Retrieval Practice — Lab 1

Pause here and complete Lab 1, then check its solution.

glasspaper oblicum

© Rick Beerendonk

Agent Service

glasspaper



© Rick Beerendonk

What is an AI Agent?

[EXAM]

Problem: LLMs respond to a single prompt; they cannot take multi-step actions

Agent = model + instructions + tools + automatic tool calling

Agent types in Microsoft Foundry:

Type	Description
Prompt agent	Model + instructions + tools, defined by configuration and run by Foundry (most common)
Voice-based prompt agent	A prompt agent for real-time spoken conversations
Hosted agent	Your own code in a container, hosted by the Foundry platform

Not an agent type any more: workflows built in the portal designer are being retired in favour of the Microsoft Agent Framework.

© Rick Beerendonk

Agent Service Features

Feature	Description
Automatic tool calling	Agent decides when and how to invoke tools
Securely managed data	Conversation data managed via Responses API
Extensive tool catalog	Functions, code interpreter, file search, web search, MCP servers
Enterprise security	Microsoft Entra identity, RBAC, content filters
Customizable storage	Basic setup: managed by Microsoft; standard setup: your own Azure resources
Observability/tracing	Tracing, metrics, evaluations; Application Insights integration
Model selection	Models from the Foundry model catalog

Required resources: Foundry project + model deployment
Optional: Azure AI Search, Azure Storage, Azure Key Vault, Azure Functions

© Rick Beerendonk

Development Approaches

Approach	Best for
Foundry portal	No-code visual design, playground testing
Foundry Toolkit for VS Code	Code-first, YAML, version control

VS Code Agent YAML pattern:

```yaml
# yaml-language-server: $schema=https://aka.ms/ai-foundry-vsc/agent/1.0.0
version: 1.0.0
name: my-agent
model:
  id: "gpt-5.2"
instructions: |
  You are a helpful assistant.
tools: []
```


© Rick Beerendonk

Create Agent via SDK

[EXAM]

```python
from azure.ai.projects import AIProjectClient
from azure.ai.projects.models import PromptAgentDefinition

project_client = AIProjectClient(
    credential=DefaultAzureCredential(),
    endpoint="https://{resource-name}.services.ai.azure.com/api/projects/{project-name}"
)

agent = project_client.agents.create_version(
    agent_name="my-agent",
    definition=PromptAgentDefinition(
        model="gpt-5.2",
        instructions="You are a helpful assistant.",
        tools=[],
    )
)

agent = project_client.agents.get(agent_name="my-agent")   # retrieve an existing agent
```

Keyless access: `DefaultAzureCredential()` plus the **`get`** operation retrieves an existing agent through Entra ID. An API-key credential is not Entra ID, and `create` or `delete` do not return the existing agent.

glasspaper oblicum

© Rick Beerendonk

Generation Controls

[EXAM]

Control	Main effect	Exam consideration
Reasoning effort: `reasoning_effort` (Chat Completions) / `reasoning={"effort": ...}` (Responses API)	How much the model thinks before it answers	Lower for simple, fast tasks; higher for multi-step problems. More effort means more reasoning tokens, latency, and cost
Output limit: `max_output_tokens` (Responses API) / `max_completion_tokens` (Chat Completions), formerly `max_tokens`	Output budget, for reasoning tokens and the visible answer together	Affects completeness and cost

Do not use a parameter to solve the wrong problem: grounding fixes missing knowledge; more reasoning effort does not add facts.

Reasoning models are controlled with reasoning effort. With reasoning switched on, `temperature` and `top_p` are rejected.

© Rick Beerendonk

Generation Controls — Reasoning Effort and Output Limit in Code

[EXAM]

`gpt-5.2` and `gpt-5-mini` are reasoning models. The values of reasoning effort vary by model and include `none`, `minimal`, `low`, `medium`, and `high`.

```
response = client.responses.create(
    model="gpt-5.2",
    input="Is it snowing in Oslo, and should I cancel the trip?",
    reasoning={"effort": "low"},
    max_output_tokens=500,
)
```

Answer cut short: the output limit was reached. The Responses API returns `status: "incomplete"` with reason `max_output_tokens`; Chat Completions returns finish reason `length`. Raise the limit; it must hold the reasoning tokens as well as the answer.

glasspaper oblicum

© Rick Beerendonk

Retrieval Practice — Lab 2

Pause here and complete Lab 2, then check its solution.

glasspaper oblicum

© Rick Beerendonk

Conversations, Responses, and Memory

glasspaper



© Rick Beerendonk

Agent Lifecycle and Runtime State

[EXAM]

Agent: instructions, model, and tools

Conversation: persistent history across turns

Item: a message, tool call, or tool output in the conversation

Response: one execution against the conversation; it produces output items

Use a durable conversation or persisted message store when history must survive sessions.

To keep a full case history across sessions, store the conversation ID and reuse it. Saving only the final model response loses the tool calls and tool outputs; summarizing condenses history instead of restoring the complete execution record.

© Rick Beerendonk

Agent Service Lifecycle — Conversation and Response

[EXAM]

For the exam, distinguish the persistent conversation from one execution:

```
Agent          = instructions + model + tools
Conversation   = persistent history of items
Item           = message, tool call, or tool output in a conversation
Response       = one execution of an agent against a conversation
```

Why it matters: a conversation stores context, while a response represents the model and tool execution that produces the next output.

Classic API: threads and runs are the names of conversations and responses in the classic API.

© Rick Beerendonk

Lifecycle in Code — Agent, Conversation, Response

[EXAM]

Typical flow: create the agent once, open a conversation, then ask for a response.

```
agent = project_client.agents.create_version(
    agent_name="my-agent",
    definition=PromptAgentDefinition(model="gpt-5.2", instructions="You are a helpful assistant."),
)

conversation = openai_client.conversations.create()

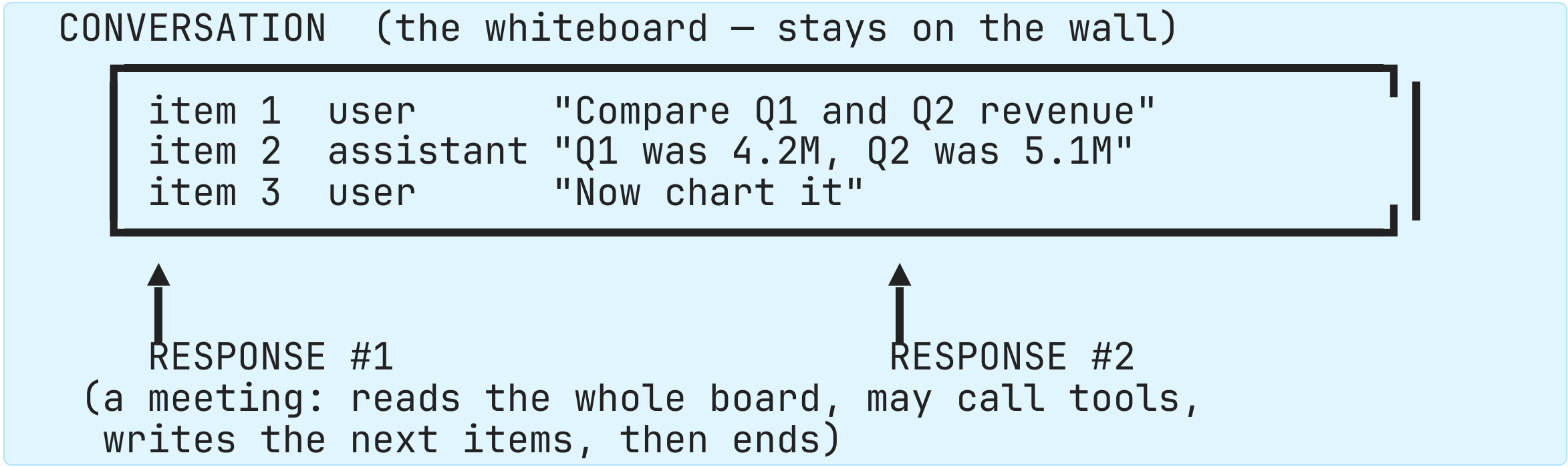
response = openai_client.responses.create(
    conversation=conversation.id,
    input="Compare Q1 and Q2 revenue",
    extra_body={"agent_reference": {"name": agent.name, "type": "agent_reference"}},
)
print(response.output_text)
```

A second `responses.create()` with the same `conversation.id` continues the same history.

glasspaper oblicum

© Rick Beerendonk

Conversation vs Response — the Whiteboard and the Meeting



The conversation persists; a response is over as soon as it produces its output. Cost, latency, tool calls, and failures all belong to a *response*. Context belongs to the *conversation*.

glasspaper oblicum

© Rick Beerendonk

Agent Memory and Scopes

[EXAM]

Memory is not RAG and not conversation history.

Feature	Stores
Conversation	The full turn history of one ongoing case
Knowledge	Source documents retrieved as evidence
Memory	Facts and preferences reused in *new* conversations

`scope` is a partition key. It decides who shares a memory:

`scope` value	Who shares the memory
`"{{$userId}}"`	One signed-in user, across conversations — no ID in code
A static value, such as `"support-team"`	Everyone who uses that value — it leaks between users when the memories are personal

Lifetime: a default time to live (TTL) on the memory store decides how long memories are kept. The scope does not.

glasspaper oblicum

© Rick Beerendonk

Memory Scope in Code — `{{$userId}}`

[EXAM]

Attach the memory tool with a scope. Use the placeholder, not a user ID that you build yourself.

```
memory_tool = MemorySearchPreviewTool(
    memory_store_name="support_memory",
    scope="{{$userId}}",
    update_delay=300,
)

agent = project_client.agents.create_version(
    agent_name="support-agent",
    definition=PromptAgentDefinition(model="gpt-5.2", tools=[memory_tool]),
)
```

The `{{$userId}}` placeholder is substituted at runtime, so each user is isolated without generating identifiers yourself. The value comes from the `x-memory-user-id` request header or, without it, from the caller's Microsoft Entra identity.

Before this works: create the memory store (`memory_store_name`) with a chat model and an embedding model. `update_delay` is the number of seconds without activity before memories are written.

© Rick Beerendonk

Retrieval Practice — Lab 3

Pause here and complete Lab 3, then check its solution.

glasspaper oblicum

© Rick Beerendonk

Custom Tools

glasspaper



© Rick Beerendonk

Custom Tool Options

[EXAM]

Option	Description
Custom function (function calling)	Describe structure; agent decides when to invoke
Azure Functions	Event-driven via queue triggers; input/output bindings
OpenAPI specification	Connect to any HTTP API via an OpenAPI 3.0 or 3.1 spec
MCP server	Tools published by a server and shared by many agents (next part)

Azure Logic Apps: the low-code action is documented only for classic agents, which retire on 31 March 2027.

© Rick Beerendonk

Function Calling

```python
from azure.ai.projects.models import FunctionTool, PromptAgentDefinition

function_tool = FunctionTool(
    name="recent_snowfall",
    parameters={
        "type": "object",
        "properties": {
            "location": {"type": "string", "description": "City name"},
        },
        "required": ["location"],
        "additionalProperties": False
    },
    description="Get recent snowfall totals for a given location.",
    strict=True,
)
```

`strict=True` enforces the parameter schema

glasspaper oblicum

© Rick Beerendonk

Function Calling — Attach the Tool

Pass the tool in the `tools` list of the agent definition. The model then decides when to call it.

```
agent = project_client.agents.create_version(
    agent_name="snowfall-agent",
    definition=PromptAgentDefinition(
        model="gpt-5.2",
        instructions="Help users find snowfall data.",
        tools=[function_tool],
    )
)
```

glasspaper oblicum

© Rick Beerendonk

Azure Functions Tool

```python
from azure.ai.projects.models import (
    AzureFunctionTool, AzureFunctionDefinition,
    AzureFunctionBinding, AzureFunctionStorageQueue
)

tool = AzureFunctionTool(
    azure_function=AzureFunctionDefinition(
        input_binding=AzureFunctionBinding(
            storage_queue=AzureFunctionStorageQueue(
                queue_name="INPUT_QUEUE",
                queue_service_endpoint="STORAGE_QUEUE_ENDPOINT",
            )
        ),
        output_binding=AzureFunctionBinding(
            storage_queue=AzureFunctionStorageQueue(
                queue_name="OUTPUT_QUEUE",
                queue_service_endpoint="STORAGE_QUEUE_ENDPOINT",
            )
        ),
        function=AzureFunctionDefinitionFunction(
            name="queue_trigger",
            description="Process a request",
            parameters={...},
        ),
    )
)
```

Communication via Storage Queue (input + output bindings)

glasspaper oblicum

© Rick Beerendonk

OpenAPI Tool

Supports OpenAPI 3.0 and 3.1. Each operation needs an `operationId`.

```
from azure.ai.projects.models import OpenApiTool, OpenApiAnonymousAuthDetails

tool = OpenApiTool(
    openapi=OpenApiFunctionDefinition(
        name="get_weather",
        spec=openapi_weather,     # dict loaded from JSON/YAML file
        description="Get weather data",
        auth=OpenApiAnonymousAuthDetails(),
    )
)
```

Supported auth types: `anonymous`, `API key`, `managed identity` (the project's managed identity)

glasspaper oblicum

© Rick Beerendonk

OpenAPI Authentication and Connections

[EXAM]

When the API needs a key on every call, declare an **API-key security scheme** in the OpenAPI spec so the platform attaches it automatically:

```
from azure.ai.projects.models import (
    OpenApiProjectConnectionAuthDetails, OpenApiProjectConnectionSecurityScheme
)

auth = OpenApiProjectConnectionAuthDetails(
    security_scheme=OpenApiProjectConnectionSecurityScheme(
        project_connection_id=project_connection.id
    )
)
```

The key stays in the project connection; the agent definition only refers to it.

© Rick Beerendonk

OpenAPI Authentication — Which Approach?

[EXAM]

Approach	Result
API-key security scheme	Key injected on every request — defined once
Project connection	Reuses the key already stored in the project connection
Adding the header manually	Repeats secret handling in code and risks leaking the key
Bearer/OAuth scheme	Expects an OAuth token, not an API key
Identity passthrough	Forwards the **caller's identity**, not the stored API key

© Rick Beerendonk

Project Connections — One Credential, Many Agents

[EXAM]

Problem: every agent and every application configures the Azure AI Search key for itself.

Solution: add a **connection** to the Search resource in the Foundry project, then reference that connection from each agent and app.

Approach	Result
Project connection to Azure AI Search	One place to store, rotate, and audit the credential
A custom HTTP connection per app	Duplicated configuration and administration
One managed identity per app	Duplicated role assignments, still configured per app
Calling Search directly from each app	The search configuration is never shared

Not credential management: enabling RBAC and disabling key-based access **hardens** the resource; a managed private endpoint controls **network** access. Neither centralizes the credential.

Isolation still applies: give each agent its **own identity** for tool and data access, so permissions can be granted, audited, and revoked per agent.

glasspaper oblicum

© Rick Beerendonk

MCP Tools

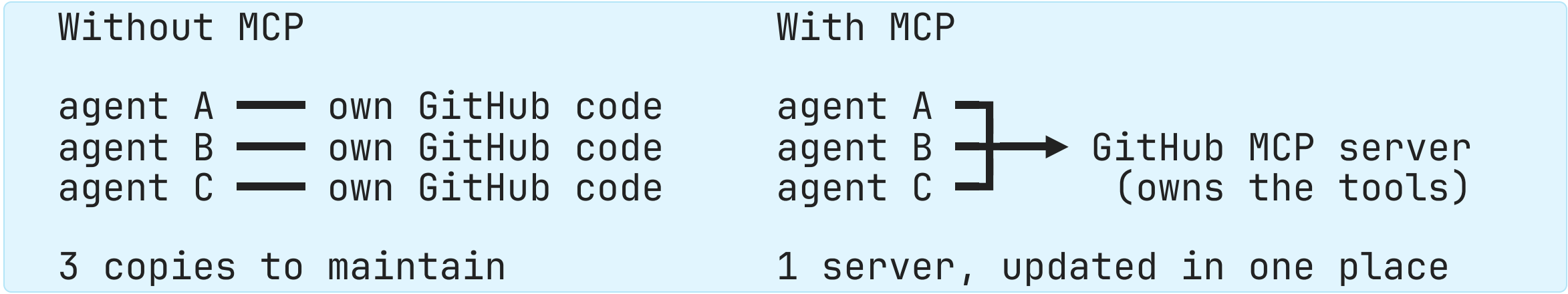
glasspaper

oblicum

© Rick Beerendonk

Why MCP Exists

Problem: every tool you hand-code lives inside one agent. Ten agents that need the same GitHub tools means ten copies to build, secure, and update.



MCP = Model Context Protocol. A standard way for a *server* to publish tools and for any agent to consume them.

Mental model: MCP is a power socket. The agent brings the plug; it does not need to know how the power station works.

glasspaper oblicum

© Rick Beerendonk

MCPTool — Remote MCP Servers

[EXAM]

No manual session/client management needed when using built-in `MCPTool`:

```
from azure.ai.projects.models import MCPTool, PromptAgentDefinition

mcp_tool = MCPTool(
    server_label="GitHub",
    server_url="https://api.githubcopilot.com/mcp/",
    allowed_tools=["list_repos"],    # optional: restrict accessible tools
    require_approval="never",        # or "always"
    headers={"Authorization": "Bearer {token}"},
)

agent = project_client.agents.create_version(
    agent_name="github-agent",
    definition=PromptAgentDefinition(
        model="gpt-5.2",
        instructions="Help with GitHub tasks.",
        tools=[mcp_tool],
    )
)
```

Multiple MCP servers supported — add each as a separate `MCPTool`

glasspaper oblicum

© Rick Beerendonk

MCP Tool Approval

[EXAM]

`require_approval="always"` → agent response contains `mcp_approval_request`

```python
# Check for approval requests in the response
approvals = []
for item in response.output:
    if item.type == "mcp_approval_request":
        # Present to user or auto-approve
        approvals.append(
            McpApprovalResponse(
                type="mcp_approval_response",
                approval_request_id=item.id,
                approve=True,
            )
        )

# Send the decisions in the same conversation
response = openai_client.responses.create(conversation=conversation.id, input=approvals)
```

glasspaper oblicum

© Rick Beerendonk

MCP vs. Hard-Coded Tools

Key advantages of MCP vs. hard-coded tools:

- **"Integrate once"** — tools updated on server; no agent redeployment needed
- **LLM-agnostic** — switch underlying model without reworking integrations
- **Standardized auth** — consistent authentication mechanism across tools

© Rick Beerendonk

Retrieval Practice — Lab 4

Pause here and complete Lab 4, then check its solution.

glasspaper oblicum

© Rick Beerendonk

Choosing and Controlling Tools

glasspaper



© Rick Beerendonk

Tool Taxonomy

[EXAM]

Tool	Primary capability	Typical control
Function tool	Application-owned operation	Your schema, validation, and execution
MCP tool	Discoverable remote tools	Server permissions and approval policy
File search	Retrieval from uploaded files or vector stores	Vector store and citation controls
Code interpreter	Sandboxed Python execution	Files, timeout, memory, and network limits
Web/Bing grounding	Current external information	Search scope, citations, and source quality
Azure Function/OpenAPI	Connect APIs and event-driven logic	Authentication and API contract

Selection rule: use the narrowest tool that solves the task, and require approval for actions with external side effects.

© Rick Beerendonk

Controlling Tool Use — `tool_choice`

[EXAM]

By default the model decides. When an agent sometimes answers **without** calling its knowledge tool, the citations are ungrounded — force the call.

Value	Behavior
`tool_choice="auto"`	Default — the model may skip every tool
`tool_choice="required"`	Must call **at least one** tool before answering
`tool_choice={"type": "function", "name": "recent_snowfall"}`	Must call **that specific** tool
`tool_choice="none"`	May not call any tool

Required vs specific: `"required"` guarantees *a* tool runs; it does not say *which*. To force one tool, name it, as in the third row.

© Rick Beerendonk

`tool_choice` in a Request

Pass `tool_choice` with the request, next to the tools it applies to.

```
response = client.responses.create(
    model=DEPLOYMENT,
    input=messages,
    tools=[mcp_tool],
    tool_choice="required",
)
```

With `"required"`, the model cannot answer before at least one of the tools has run.

© Rick Beerendonk

Human Approval Contract

[EXAM]

When a tool call requires approval, the SDK submits a Boolean decision such as `approve=True` or `approve=False` tied to the tool call. The string `"approved"` is not the SDK approval value.

Two contexts, two contracts. Both are correct in their own place:

Context	What pauses	Approval value
SDK tool approval (`require_approval="always"`)	One tool call, with an `mcp_approval_request`	Boolean: `approve=True` or `approve=False`
Workflow YAML (`ask_question` step, see Agent Workflows)	The whole workflow, until a person answers	A workflow variable with the string `"approved"`

Exam check: read which context the question describes before you choose `approve=True` or `"approved"`.

glasspaper oblicum

© Rick Beerendonk

Retrieval Practice — Lab 5

Pause here and complete Lab 5, then check its solution.

glasspaper oblicum

© Rick Beerendonk

Foundry IQ — Knowledge

glasspaper



© Rick Beerendonk

What is Foundry IQ?

[EXAM]

Microsoft's managed knowledge platform for AI agents, built on Azure AI Search

- Multiple agents share the same knowledge bases
- Uses MCP internally to connect agents to knowledge bases

RAG steps: Retrieve → Augment → Generate

Knowledge sources are indexed (ingested before query time) or remote (queried live):

Knowledge source	Indexed or remote	Best for
Search index	Indexed	Existing search investment
Azure blob	Indexed	Files in Azure Storage
OneLake	Indexed	Microsoft Fabric data
Indexed SharePoint (preview)	Indexed	Advanced search, custom pipelines
Remote SharePoint (preview)	Remote	Live SharePoint
Web (Bing)	Remote	Public, current information

Also, all in preview: Azure SQL, File, Fabric data agent, Fabric ontology, MCP server, Work IQ.

© Rick Beerendonk

Connecting an Agent to a Knowledge Base

[EXAM]

```python
from azure.ai.projects.models import MCPTool, PromptAgentDefinition

knowledge_tool = MCPTool(
    server_label="product-docs",
    server_url=f"{search_endpoint}/knowledgebases/product-documentation/mcp?api-version=2026-08-01-preview",
    require_approval="never",
    allowed_tools=["knowledge_base_retrieve"],
    project_connection_id=project_connection_name,
)

agent = project_client.agents.create_version(
    agent_name="product-support-agent",
    definition=PromptAgentDefinition(
        model="gpt-5-mini",
        instructions="Answer product questions. Always cite your sources.",
        tools=[knowledge_tool]
    )
)
```

The Search credential comes from the **project connection**, not from application code.

© Rick Beerendonk

Azure AI Search RBAC

[EXAM]

For read-only agent queries against a shared Azure AI Search service, assign the agent identity the **Search Index Data Reader** role. Managed identities let each agent receive a separate, auditable data-plane permission.

Role boundaries:

- **Search Index Data Reader:** query documents in indexes
- **Search Index Data Contributor:** read and push documents
- **Search Service Contributor:** manage indexes, indexers, and data sources

Use the narrowest data-plane role that satisfies the agent's task; do not grant service-management permissions to a retrieval-only agent.

© Rick Beerendonk

Retrieval Practice — Lab 6

Pause here and complete Lab 6, then check its solution.

glasspaper oblicum

© Rick Beerendonk

Classic RAG vs Agentic RAG

[EXAM]

Classic RAG searches once with the user's words, then answers.

```
question ---> one search ---> generate
```

Agentic RAG gives retrieval to the model as a tool, so it plans the search itself.

```
question ---> rewrite / split into sub-questions
                  |  searches run in parallel
                  v
              read results ---> not enough? search again ---> generate
```

Choose	When
Classic RAG	One index, FAQ-style questions, fixed cost and latency
Agentic RAG	Answer needs **several chunks**, earlier turns must steer retrieval, or sources must be searched in parallel

Not the same as: *iterative retrieval* (repeats a query without conversation-aware planning) or *chain of thought* (structures reasoning, not retrieval).

© Rick Beerendonk

Retrieval Instructions Pattern

Three critical behaviors to specify:

```
retrieval_instructions = """
CRITICAL RULES:
- You must ALWAYS search the knowledge base before answering any question
- You must NEVER answer from your own knowledge or training data
- Every answer must include citations in this format:
   【message_idx:search_idx†source_name】
- If the knowledge base doesn't contain the answer, respond with:
  "I don't have that information in my knowledge base."
"""
```

glasspaper oblicum

© Rick Beerendonk

Retrieval Practice — Lab 7

Pause here and complete Lab 7, then check its solution.

glasspaper oblicum

© Rick Beerendonk

Microsoft 365 Integration

glasspaper



© Rick Beerendonk

Every Agent Has an Endpoint and an Identity

[EXAM]

From the moment an agent is created, it has:

- A stable endpoint (the URL stays the same across version updates)
- Its own agent identity in Microsoft Entra ID

The agent identity starts with no permissions. Assign Azure RBAC roles to it on each resource its tools use (AI Search, Storage). Foundry assigns none automatically.

Active version: "Always use latest" by default, so a new version is served at once. Pin a specific version to protect production.

Not the same as: an *Agent Application* (the earlier publishing model) or a *legacy agent* (shares the project identity).

© Rick Beerendonk

Publishing an Agent to Teams/Copilot

[EXAM]

Publishing distributes the agent to Microsoft Teams and Microsoft 365 Copilot. Foundry handles the Teams bot registration and builds the app package.

Publishing scopes:

Scope	Visibility	Requires admin approval
Just you (formerly Shared)	"Your agents" in Teams	No — immediate
People in your organization (formerly Organization)	"Built by your org" in Teams	Yes — in the Microsoft 365 admin center

Required roles: Foundry User on the project (publish), plus the right to create a Bot Service resource in the resource group; Foundry Agent Consumer (only call an agent)

`Microsoft.BotService` provider must be registered in the subscription

Permissions: roles go to the agent identity. There is no "reassign after publishing" step.

© Rick Beerendonk

Microsoft 365 Publishing Options

Direct Foundry publishing:

- Publishes the agent's stable endpoint to Teams and Microsoft 365 Copilot
- Handles the Teams bot registration; Foundry creates the Azure Bot Service resource
- Generates a Microsoft 365 app package
- Serves new agent versions through the same endpoint, without republishing

Other channels:

Channel	Use case
Teams and Microsoft 365 Copilot	Users work inside Microsoft 365
Web application preview	Browser-based demonstrations and stakeholder testing
Stable API endpoint	Embed the agent in a custom application
Azure Bot Service channels	Slack, Telegram, Twilio, Facebook, and other channels

Use the Microsoft 365 Agents Toolkit when the solution needs custom single sign-on, middleware, or multi-environment deployment.

© Rick Beerendonk

Access Microsoft 365 Data with Work IQ

Work IQ gives an agent access to authorized workplace context such as:

- Emails and conversations
- Meetings and calendar information
- Documents and collaboration data

Important boundaries:

- The signed-in user's Microsoft 365 permissions still apply
- Request only the data needed for the task
- Explain which data sources the agent can use
- Validate citations and permissions before exposing results

Work IQ is different from Foundry IQ:

	Foundry IQ	Work IQ
Primary data	Configured enterprise knowledge sources	Microsoft 365 workplace data
Typical use	Product, policy, and document knowledge	Personal and organizational work context
Integration	MCP-based knowledge connection	Work IQ MCP endpoint; the built-in tool is in preview

© Rick Beerendonk

Test and Operate a Published Agent

Before publishing:

1. Test normal, ambiguous, and adversarial prompts in the Foundry playground
2. Verify every configured tool and knowledge source
3. Check citations, authentication, and user-data isolation
4. Confirm the agent identity has the required Azure RBAC roles

After publishing:

- Test in Teams and Microsoft 365 Copilot with representative users
- Monitor invocation failures, latency, tool calls, and user feedback
- Update the agent version without changing the stable endpoint; pin the active version when production must not change yet
- Assign roles to the agent identity of every new agent

Common failure: a tool fails with an authorization error because the agent identity has no role on the resource. A new agent identity starts without permissions.

glasspaper oblicum

© Rick Beerendonk

Agent Workflows

glasspaper

oblicum

© Rick Beerendonk

Workflow Patterns

[EXAM]

For a workflow that must pause before an external side effect, use an `ask_question` step and continue only when the returned value equals `"approved"`. This is a workflow-variable approval pattern, distinct from the SDK's typed `approve=True` tool-approval contract. In the portal designer this step is the **Ask a question** node.

Retirement: workflows in the Foundry portal retire on 1 December 2026. Build new workflows with the Microsoft Agent Framework.

Pattern	Description
Sequential	Fixed step-by-step pipeline; predictable, ordered
Human-in-the-loop	Pauses to request user input or approval before resuming
Group chat	Dynamic multi-agent orchestration; control shifts between agents

glasspaper oblicum
© Rick Beerendonk

Workflow Node Types

A workflow is a set of connected nodes. Each node does one kind of work:

Node	Sub-types / Purpose
Invoke	Calls an AI agent; returns free-text or structured JSON
Flow	`If/Else`, `Go To`, `For Each` — control execution path
Data transformation	`Set Variable`, `Reset Variable`, `Parse value`
Basic chat	Sends/receives messages from the user
End	Terminates workflow; optionally returns final result

© Rick Beerendonk

Choosing an Orchestration

[EXAM]

Requirement	Choose
Deterministic steps, conditions, and shared state with minimal development effort	**A workflow**
Ordered work with approval before a risky update	**Sequential template + an Ask a question node**
Agents decide who contributes next	Group chat
Independent steps that can run at the same time	Concurrent

Why not the alternatives: a multi-agent group chat is built for autonomous conversational collaboration and lets the agents choose the order; concurrent orchestration runs steps in parallel; conversations and responses without a workflow — or separate agent runs coordinated in your own application code — mean you write and maintain the orchestration yourself.

Retirement: workflows in the Foundry portal retire on 1 December 2026. After that date, build the same shapes with the Microsoft Agent Framework.

© Rick Beerendonk

Power Fx Formulas in Workflows

[EXAM]

Logic in workflow nodes:

```
Upper(Local.Input) / Lower(Local.Input) / Len(Local.Input)
Local.Confidence > 0.8                  → boolean condition
If(Local.Confidence > 0.8, "Proceed", "Escalate")
Sum(Local.ItemList, Amount)
Count(Local.ItemList)
IsBlank(Local.Input) / IsEmpty(Local.ItemList)
ForAll(Local.ItemList, Upper(Name))
Concatenate(Local.FirstName, " ", Local.LastName)
```

Variables: `Local.*` (captured during execution), System variables (workflow context)

Interpolation in message text: wrap the expression in braces — `{Upper(Local.Var01)}`. Without braces the formula is printed as literal text.

glasspaper oblicum

© Rick Beerendonk

Power Fx — variable scopes & operators

Scope prefix is required — bare `Var1` does not resolve.

Prefix	Source
`Local.<name>`	Workflow variables you defined
`System.<name>`	Built-in, supplied by the platform (e.g. `System.Conversation.Id`)

Boolean operators: `&&` `||` `Not(...)` `=` `<>` `>` `<` `>=` `<=`

```
Not(IsBlank(Local.Email)) && Local.Age >= 18
Local.Confidence > 0.8 || Local.Override = true
```

glasspaper oblicum

© Rick Beerendonk

If/Else node — "must have a value"

[EXAM]

The **If/Else node** takes a **boolean expression** (not a value-returning `If(...)`).

Goal	Condition
Variable **must have a value**	`Not(IsBlank(Local.Var1))`
Variable is missing	`IsBlank(Local.Var1)`
List has at least one item	`Not(IsEmpty(Local.ItemList))`
Two values both required	`Not(IsBlank(Local.A)) && Not(IsBlank(Local.B))`

`IsBlank` vs `IsEmpty`:

- `IsBlank(x)` — true when `x` is `null` **or** an empty string `""`
- `IsEmpty(coll)` — true when a **collection / table** has zero rows

> The function `If(cond, a, b)` **returns a value** — use it inside *Set Variable*, not as the condition of an If/Else node.

© Rick Beerendonk

Invoke a Workflow from Code

```
conversation = openai_client.conversations.create()

stream = openai_client.responses.create(
    conversation=conversation.id,
    extra_body={"agent_reference": {"name": "triage-workflow", "type": "agent_reference"}},
    input="Users can't reset their password from the mobile app.",
    stream=True,
)

for event in stream:
    if event.type == "response.completed":
        print(event.response.output_text)
```

Check action status: in `response.output`, each item with `type == "workflow_action"` carries a `status`.

glasspaper oblicum

© Rick Beerendonk

Microsoft Agent Framework

glasspaper oblicum

© Rick Beerendonk

Framework Overview

Open-source SDK; provider-agnostic (Foundry, Azure OpenAI, OpenAI, Anthropic, Amazon Bedrock, Ollama)

vs. Foundry Agent Service:

	Microsoft Agent Framework	Foundry Agent Service
Compute	Your infrastructure	Fully managed
Conversation history	In the session object, or service-side with a Foundry or Responses provider	Service-side
Providers	Any (OpenAI, Anthropic, …)	Foundry models
Control	Full code control	Configuration-based

glasspaper oblicum

© Rick Beerendonk

Core Components

Component	Description
`Agent`	Main class: client + instructions + tools
`FoundryChatClient`	Connection + auth to Foundry project (from `agent_framework.foundry`)
`AgentSession`	Persistent conversation history (roles: USER, ASSISTANT, SYSTEM, TOOL)
Built-in tools	Code Interpreter, File Search, Web Search
`@tool` **decorator**	Marks Python functions as agent tools

glasspaper oblicum

© Rick Beerendonk

Custom Tools with @tool

```python
from agent_framework import Agent, tool
from agent_framework.foundry import FoundryChatClient
from pydantic import Field
from typing import Annotated

@tool(name="get_stock_price", description="Get current price for a stock ticker")
def get_stock_price(
    ticker: Annotated[str, Field(description="Stock ticker symbol, e.g. MSFT")]
) -> float:
    return 428.50

agent = Agent(
    client=FoundryChatClient(...),
    instructions="You are a financial advisor.",
    tools=[get_stock_price]
)
session = agent.create_session()
result = await agent.run("What is the price of MSFT?", session=session)
```

Use `Annotated` + `Field` from Pydantic for parameter descriptions

glasspaper oblicum

© Rick Beerendonk

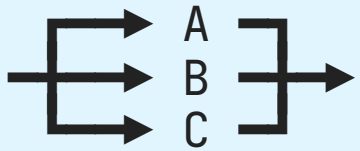
Multi-Agent Orchestration

glasspaper oblicum

© Rick Beerendonk

Orchestration Patterns — the Shapes

Each pattern is really a picture of *who talks to whom, in what order*.

Sequential	A → B → C	each one refines the last
Concurrent		all at once, then combine
Group chat	A ↔ B ↔ C	a manager picks who speaks next
Handoff	A →?→ B or C	A decides who should take over
Magentic	plan → A → replan → C	plan first, adapt as it goes

Ask two questions to pick one: Do the steps depend on each other? Is the order known before you start?

© Rick Beerendonk

Orchestration Patterns

[EXAM]

Pattern	Builder class	When to use	Avoid when
Sequential	`SequentialBuilder`	Step-by-step pipelines; progressive refinement	Stages can run in parallel
Concurrent	`ConcurrentBuilder`	Parallel, independent tasks; ensemble; voting	Tasks depend on each other
Group chat	`GroupChatBuilder`	Collaborative conversation; maker-checker; human-in-the-loop	Simple linear tasks; speed critical
Handoff	`HandoffBuilder`	Dynamic delegation; expertise unknown upfront	Fixed agent order known
Magentic	`MagenticBuilder`	Complex open-ended; plan-first; dynamic task ledger	Simple/deterministic tasks

All patterns: `Builder(participants=[...])` → `build()` → `run(...)` or `run(..., stream=True)`
Results: `get_outputs()` → iterate messages with `.author_name` and `.text`

Sequential output: only the last participant by default; name earlier ones in `intermediate_output_from` to receive their `"intermediate"` events.

Import: all five builders come from `agent_framework.orchestrations`.

© Rick Beerendonk

Workflow Edge Types

Edge	Description
Direct	Sequential: A → B
Conditional	Execute only when condition is met
Switch-Case	Route to different executors based on predefined cases
Fan-Out	Send one message to multiple executors simultaneously
Fan-In	Combine outputs from multiple executors into one

Built-in events: one `WorkflowEvent` class with a `type`: `"started"`, `"output"`, `"failed"`, `"executor_invoked"`, `"executor_completed"`, `"request_info"`

Workflows support **checkpointing** (save/resume state)

© Rick Beerendonk

Retrieval Practice — Lab 8

Pause here and complete Lab 8, then check its solution.

glasspaper oblicum

© Rick Beerendonk

A2A Protocol

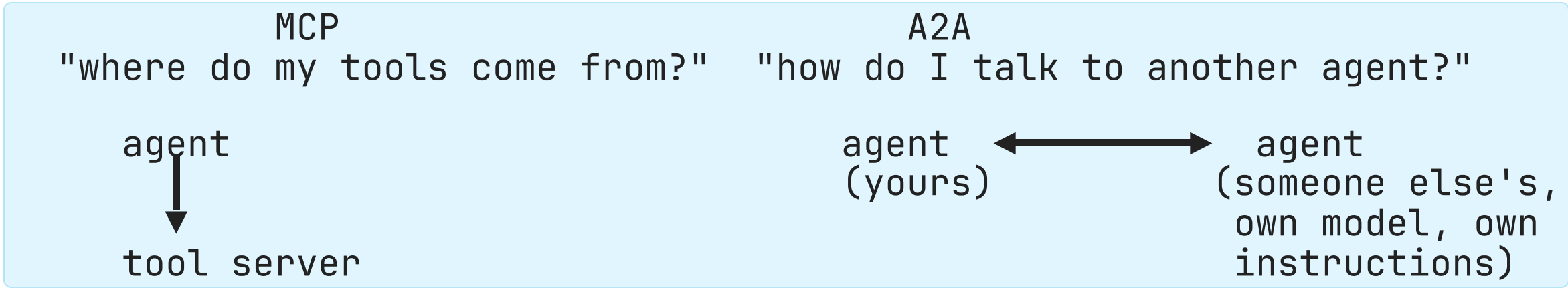
glasspaper

oblicum

© Rick Beerendonk

MCP and A2A Are Not Competitors

They solve two different questions.



Question	MCP	A2A
Other side is…	a tool provider	another autonomous agent
Who picks the model?	you, for your agent	each agent picks its own
Discovery	server URL + tool list	`agent-card.json`

One sentence: MCP gives an agent *hands*; A2A gives it *colleagues*.

glasspaper oblicum

© Rick Beerendonk

Agent-to-Agent (A2A)

Standardized framework for agent discovery, communication, and coordinated task execution

Key advantage over MCP: Each A2A agent chooses its own LLM; integrated authentication; cross-vendor

In Foundry: an agent's own endpoint can expose the A2A protocol (`…/agents/<agent>/endpoint/protocols/a2a`) and an agent card, so a Foundry agent can be the remote agent as well as the caller.

Agent Card — discovery document at `/.well-known/agent-card.json`:

- `name` and `description`: who the agent is
- `supportedInterfaces`: where to reach it (URL, protocol binding, protocol version)
- `capabilities`: for example streaming
- `skills`: what it can do, with tags and example prompts

glasspaper oblicum

© Rick Beerendonk

A2A Agent Card — Example

```
{
  "name": "weather-agent",
  "description": "Provides weather forecasts",
  "supportedInterfaces": [
    {
      "url": "https://my-agent.example.com/",
      "protocolBinding": "JSONRPC",
      "protocolVersion": "1.0"
    }
  ],
  "capabilities": { "streaming": true },
  "skills": [
    {
      "id": "weather-forecast",
      "name": "Weather Forecast",
      "description": "Get weather for a location",
      "tags": ["weather", "forecast"],
      "examples": ["What is the weather in Paris?"]
    }
  ]
}
```

glasspaper oblicum

© Rick Beerendonk

A2A Server and Client

Server components:

Component	Role
Agent Card	Describes capabilities; at `/.well-known/agent-card.json`
Request Handler	Routes requests to `AgentExecutor`; manages Task Store
Server Application	Starlette + Uvicorn (ASGI); listens on HTTP

AgentExecutor interface:

- `execute` — processes requests; sends results via `EventQueue`
- `cancel` — handles task cancellation

Client request types:

- **Non-streaming** — send message, wait for complete response
- **Streaming** — incremental responses for long-running tasks

glasspaper oblicum

© Rick Beerendonk

Agent Evaluation and Operations

Find the part that failed, then measure, correct, and monitor it.

glasspaper



© Rick Beerendonk

Problem: "The Agent Gave a Bad Answer"

That sentence is not a bug report. It could mean any of these:

- The search returned the wrong document
- The instructions never said to cite sources
- The tool threw an error and the model guessed instead
- The model is too small for the task
- Everything worked — last week's version was simply different

Need: vocabulary precise enough to say *which part* failed.

This part of the lecture gives you that vocabulary.

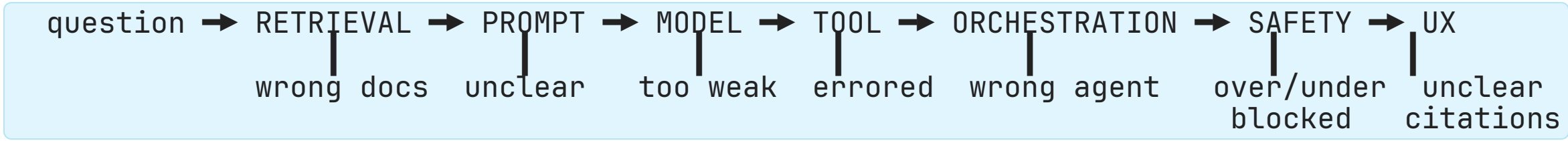
glasspaper oblicum

© Rick Beerendonk

Evaluation and Error Analysis

[EXAM]

A wrong answer has a *first* broken link. Walk the chain in order and stop at the first failure.



Use a labeled evaluation dataset, reproduce the failure, change one layer, and compare the before/after metrics.

glasspaper oblicum

© Rick Beerendonk

Error Analysis — One Question per Layer

[EXAM]

Separate failures by layer:

Failure	Diagnostic question
Retrieval	Did the index return the right source and chunk?
Prompt	Were instructions, context, and examples clear?
Model	Can the selected model perform the task?
Tool	Was the schema valid and did the tool complete?
Orchestration	Did the workflow choose the right next agent?
Safety	Should the request or response have been blocked?
User experience	Did the user understand uncertainty and citations?

Ask the questions from top to bottom. The first "no" is the layer to repair.

© Rick Beerendonk

Evaluate, Control, and Monitor Agents

[EXAM]

- Use a groundedness gate or evaluator to stop unsupported answers
- Use `Search Index Data Reader` for read-only Search access
- Use reflection to regenerate incomplete answers
- Treat `approve=True` as a typed approval decision, not a string (SDK tool approval; a workflow `ask_question` step checks for `"approved"`)
- Monitor tool calls, latency, tokens, handoffs, quality, safety, and versions

Error analysis: inspect retrieval, prompt, model, tool, orchestration, safety, and user-experience layers in order.

© Rick Beerendonk

Evaluate an Agent in a Pipeline

[EXAM]

Which evaluators for a RAG agent?

Evaluator	Answers
Retrieval	Did the search return the right documents?
Groundedness	Does the answer follow those documents?
Fluency and coherence	Readability and logical flow — not correctness

Gate a pull request with GitHub Actions:

- Authenticate with **OIDC** — federated, no long-lived stored credential
- Set the **project endpoint** so the workflow knows which Foundry project to evaluate against
- **Fail the workflow** when evaluation fails, so an unsafe version cannot merge

The evaluation config path, the model deployment name, and the tenant ID do not identify the project.

© Rick Beerendonk

Retrieval Practice — Lab 9

Pause here and complete Lab 9, then check its solution.

glasspaper oblicum

© Rick Beerendonk

Reflection and Self-Critique Loops

[EXAM]

A quality loop asks a second evaluator or agent to inspect the first answer.

```
draft answer
  → check against requirements and source context
  → identify unsupported claims or missing steps
  → revise or reject
  → return answer with evidence
```

Useful checks:

- Is every claim grounded in retrieved context?
- Does the answer satisfy the requested format?
- Did the tool call use valid and authorized arguments?
- Should a human review this result?

Trade-off: reflection can improve quality but adds latency, token cost, and another failure surface. Measure whether it changes evaluation scores.

When required clauses are missing, a reflection pass can check the draft against the requirements and regenerate the response before returning it.

© Rick Beerendonk

Retrieval Practice — Lab 10

Pause here and complete Lab 10, then check its solution.

glasspaper oblicum

© Rick Beerendonk

Production Agent Monitoring

[EXAM]

Monitor more than HTTP success:

- Run duration and time spent in each model or tool call
- Prompt, completion, and tool tokens
- Rate-limit responses and retries
- Tool-call validity, failures, and approval requests
- Groundedness, relevance, coherence, fluency, and safety scores
- Handoff routes, loop counts, and abandoned runs
- Model, prompt, agent, and knowledge-source versions

Alert on: quality regression, rising latency, cost spikes, repeated tool errors, unsafe outputs, and retrieval failures.

On Microsoft Learn: tracing and evaluation each have their own pages; this slide joins the signals into one list.

© Rick Beerendonk

Key Takeaways

1. **Agent types:** prompt agent, voice-based prompt agent, and hosted agent; portal workflows are being retired
2. **Generation controls:** reasoning effort sets how much the model thinks; the output token limit sets the budget for reasoning and answer together
3. **Conversation:** persistent history; **response:** one execution; **item:** a message, tool call, or tool output in the conversation (classic API: thread, run, message)
4. **Memory scope:** a partition key; `{{$userId}}` isolates per user, one static value is shared by everyone who uses it; a TTL on the memory store sets the lifetime
5. **Custom tools:** function calling, Azure Functions, OpenAPI 3.0 or 3.1, and MCP servers
6. **Project connections:** one shared credential for every agent; each agent still gets its own identity
7. **MCP:** `MCPTool` attaches remote MCP servers; `require_approval` controls autonomy
8. **Tools:** choose the narrowest safe tool and control external side effects
9. **`tool_choice`:** `"required"` forces *a* tool; a named tool such as `{"type": "function", "name": "recent_snowfall"}` forces *which* tool
10. **Foundry IQ:** shared knowledge bases; indexed and remote knowledge sources; citation pattern
11. **Agent Framework:** Open-source, provider-agnostic; `@tool` decorator for function tools
12. **Orchestration patterns:** Sequential, Concurrent, Group chat, Handoff, Magentic
13. **A2A:** `/.well-known/agent-card.json` for discovery; skills describe what the agent can do
14. **Error analysis:** isolate retrieval, prompt, model, tool, orchestration, safety, and UX failures
15. **Reflection:** draft, inspect, revise, and measure the added cost
16. **Operations:** monitor tokens, latency, tool calls, quality, safety, versions, and retries

© Rick Beerendonk

Appendix

How to use this lecture next to Microsoft Learn

glasspaper oblicum

© Rick Beerendonk

How to Use This Lecture Next to Microsoft Learn

Microsoft Learn explains each product feature on its own page. These slides put related facts side by side, in the way the exam asks about them.

Topic in this lecture	What the slides put together
Agent, conversation, and response lifecycle	One picture of what persists (the conversation) and what executes (a response)
Function, MCP, file search, code interpreter, and Bing grounding tools	One comparison table and a rule to choose between them
Reflection, self-critique, and error analysis	The reflection loop and a layer-by-layer search for the first failure
Agent production monitoring	One list of what to monitor and when to alert

© Rick Beerendonk